

Project: Generic Prior vs Informed Prior

Goal: Improve upon the generic prior created by Tabular In-Context Learning models by including corrosion information to make them perform better on corrosion datasets

Requirements:

- publicly available synthetic dataset creation pipeline
- publicly available model architecture
- TabICLv2
 - o Repo supports using/evaluating both TabICLv1 and TabICLv2 checkpoints, but public prior creation/pretraining code is only available for TabICLv1.
- real corrosion dataset to test
 - o [Datacortech data and code - Mendeley Data](#)
 - o Took original model code in R from authors and turned it to python; will use that for preprocessing and stuff

Current setup:

- Inside GPU cluster node:
 - o projects/tabicl (tabicl repo)
 - o projects/pfn-augmentation (own project)
 - o projects/pfn-augmentation/data/Datacortech_final_data.xlsx
- GPU nodes available: sv1221, sv1223, sv1224, sv1225
- Compute OS is Ubuntu Linux
- GPUs are NVIDIA A100 / H100 with 80 GB VRAM
- Access is via SSH, no GUI; scheduler is SLURM
 - o `srun --gpus=1 --mem=16G --time=02:00:00 --pty bash`
 - o `sbatch script.sh`

TODO general:

- clone existing publicly available Tabular In-Context to VScode
- get familiar with it
- run it to see that it works and how
- use available GPU cluster to run it
- reduce configs for efficient testing to get familiar
- train in-context model on real corrosion dataset for benchmark
- improve upon generic prior with corrosion information and test it on corrosion dataset

Done:

- cloned TabICL to GPU node

- Upgraded the Python packaging tools, installed the cloned tabicl repo into the active environment with `pip install -e .`, and verified the installation worked by successfully importing from the repo.
- Upgraded the Python packaging tools, installed the cloned tabicl repo into the active environment with `pip install -e .`, and verified the installation worked by successfully importing from the repo.
-

venv:

```
fcolanto@sv1221:~/projects/tabicl$ /usr/bin/python3.12 -m venv .venv
```

```
fcolanto@sv1221:~/projects/tabicl$ source .venv/bin/activate
```

```
(.venv) fcolanto@sv1221:~/projects/tabicl$ which python
```

```
python --version
```

```
pip --version
```

```
/home/fcolanto/projects/tabicl/.venv/bin/python
```

```
Python 3.12.3
```

```
pip 24.0 from /home/fcolanto/projects/tabicl/.venv/lib/python3.12/site-packages/pip
(python 3.12)
```

packaging tools

```
python -m pip install --upgrade pip setuptools wheel
```

```
pip install -e .
```

pip install hatch

hatch test

```
115 passed, 2 skipped, 6 warnings in 744.88s (0:12:24)
```

What I know about the project:

- repo has several stages 1 to 3, they all generate/train the model on synthetic datasets. First stage has smaller dataset sizes and last stage bigger ones.
 - Qs:
 - What are the exact differences between the stages
 - Are they called sequentially
- The repo allows for either loading a precomputed dataset or to create datasets on the fly to save memory
- Each stage creates datasets hierarchically in form of batches, groups, subgroups. Groups and subgroups share hyperparameters used to generate synthetic datasets, making them look more similar
 - Qs:
 - Which hyperparameters are shared for all batches, which for groups, which for subgroups, what do they do
 - How does this hierarchical structure actually look like
- The sampling of final features and targets is done using a SCM. Nodes represent feature vectors, edges represent mathematical transformations. Either the root nodes are sampled as features and final node as target, or nodes are randomly sampled withing the SCM graph to be assigned to feature or target.
 - Qs:
 - Does the model allow for both or only one approach
 - Does the repo create one SCM per batch, or group or subgroup or single dataset
- During generation the user can set for prior either a mlp SCM, treebased SCM, or both. When feature vector passes from one node to the other a mlp or tree transforms the data, giving the two neighboring nodes a mathematical relationship, which could later on be used to train the model to learn it.
 - Qs:
 - Are the params of mlp and trees sampled? How is this done?
 - Can there be only one prior type per SCM?
- The prior for the initial feature vectors can be either uniform, normally distributed or restricted norm distr

Lets proceed one bullet point at the time. Double check that my statements are correct by looking at the repo and paper. If so reformulate points more cleanly to save and paste into report. If incorrect fix them. Mention relevant info like script names and stuff. Ones I am happy with the first bullet point we continue to the next, after I tell you so

The model is pretrained with a three-stage curriculum that is primarily a training mechanism, although each stage also affects synthetic dataset creation because it changes the dataset-size range sampled from the prior. In the codebase, this curriculum is exposed through `scripts/train_stage1.sh`, `scripts/train_stage2.sh`, and `scripts/train_stage3.sh`, all of which call `train/run.py`; the stages are intended to run sequentially, since stage 2 loads the latest checkpoint from stage 1 and stage 3 loads the latest checkpoint from stage 2 through `--checkpoint_path` together with `--only_load_model True`. In the paper, the stages are defined as follows: stage 1 trains for 500K steps on synthetic datasets with 1,024 samples and a 30–90% train split, stage 2 trains for 40K steps on datasets with 400–10,240 samples drawn log-uniformly and an 80% train split, and stage 3 trains for 10K steps on datasets with 400–60K samples drawn log-uniformly and an 80% train split. The stage scripts implement the same progression at the code level: stage 1 uses a short fixed context (`max_seq_len 1024`), stage 2 expands to a broad log-uniform sequence-length range (`min_seq_len 1000`, `max_seq_len 40000`, `log_seq_len True`) with smaller micro-batches, and stage 3 focuses on the largest contexts (`min_seq_len 40000`, `max_seq_len 60000`), replays smaller examples (`replay_small True`), and freezes the column and row modules (`freeze_col True`, `freeze_row True`), so the final phase mainly adapts the long-context in-context-learning stage rather than relearning the full model from scratch.

The codebase supports two equivalent ways of supplying synthetic prior datasets during pretraining, which is a training/data-pipeline choice rather than part of the model architecture itself. In `src/tabc1/train/run.py`, the training entrypoint switches between these modes through `configure_prior()`: if `--prior_dir` is not set, it instantiates `PriorDataset` and generates synthetic datasets on the fly; if `--prior_dir` is set, it instantiates `LoadPriorDataset` and reads pre-generated batches from disk, with `--load_prior_start` controlling the starting batch index and `--delete_after_load` optionally removing files after they are consumed. The stage scripts explicitly show both workflows in practice: each of `scripts/train_stage1.sh`, `scripts/train_stage2.sh`, and `scripts/train_stage3.sh` first gives a direct training command that generates prior data online via `train/run.py`, and then gives an alternative two-step workflow in which `prior/genload.py` first materializes a fixed number of synthetic batches to `--save_dir` and `train/run.py` later trains from that directory via `--prior_dir`. Your original statement is therefore correct in substance, but it is more accurate to say that on-the-fly generation mainly avoids materializing a large synthetic prior corpus on disk, whereas pre-generation stores that corpus explicitly and then reuses it during training; the repo does not describe this switch primarily as a model-side memory optimization.

The batch–group–subgroup hierarchy is a synthetic dataset creation mechanism inside the prior generator, not a model-architecture feature. In `src/tabc1/prior/dataset.py`, `batch_size_per_gp` is defined as the number of datasets per group that share similar

characteristics, while `batch_size_per_subgp` is the number of datasets per subgroup with even more similar causal structures; if `batch_size_per_subgp` is not set, it defaults to `batch_size_per_gp`, so each group contains exactly one subgroup by default. The actual sharing happens at different levels: fixed configuration values in `DEFAULT_FIXED_HP` from `src/tabc1/prior/prior_config.py` are injected into every dataset and include settings such as the MLP/tree mixing probabilities, tree model family, tree-depth and tree-ensemble sampling controls, whether class generation is balanced, categorical-feature probability, maximum categorical cardinality, and whether features or labels are permuted; if `seq_len_per_gp=False`, the whole batch also shares a single sampled `seq_len` and `train_size`; each group then samples a shared higher-order hyperparameter specification through `hp_sampling()`, and `src/tabc1/prior/hp_sampling.py` makes clear that these are meta-distributions, meaning the group shares distribution parameters rather than final values; each subgroup then draws concrete values from those group-level samplers and also shares `prior_type`, `num_features`, and the realized sampled hyperparameters; finally, each individual dataset still gets its own `num_classes`. In other words, your statement is directionally right, but the precise correction is that groups do not usually share the final hyperparameter values directly; they share the hyperparameter-generating distributions, while subgroups share the realized causal/generative settings that make their datasets most similar. The structure is therefore: `batch` → `groups` → `subgroups` → `datasets`, where datasets in the same group are loosely related, datasets in the same subgroup are closely related, and every dataset is still generated independently from its own parameter dictionary. This hierarchy also looks different across stages because the training scripts set different group sizes: `scripts/train_stage1.sh` uses `batch_size=512` and `batch_size_per_gp=4`, so a batch is effectively 128 groups of 4 datasets each; `scripts/train_stage2.sh` uses `batch_size_per_gp=2` and `seq_len_per_gp=True`, so each group shares its own sequence length and train split and a batch becomes 256 groups of 2 datasets; `scripts/train_stage3.sh` uses `batch_size_per_gp=1` with `seq_len_per_gp=True`, so the hierarchy effectively collapses to one dataset per group/subgroup and almost all cross-dataset sharing inside a batch disappears. The sampled hyperparameters mainly control how the SCM prior is instantiated: `is_causal`, `num_causes`, `y_is_effect`, `in_clique`, `sort_features`, `num_layers`, `hidden_dim`, `init_std`, `noise_std`, `sampling`, and the MLP activation/dropout settings determine whether the generator behaves like a causal graph or a direct predictor, how many root causes it starts from, whether the target is sampled from late or early layers, how densely features and target are selected from intermediate states, and how deep, wide, noisy, and nonlinear the transformation network is; for the tree prior, the fixed/sampled settings determine which tree model is used and how tree depth and ensemble size are sampled before each tree layer is instantiated.

Concrete example: suppose you use the stage-1 training setup from `scripts/train_stage1.sh`, which sets `batch_size=512`, `batch_size_per_gp=4`, `prior_type=mix_scm`, `min_features=2`, `max_features=100`, `max_classes=10`, `max_seq_len=1024`, and a train split between 0.1 and

0.9. In that case, one synthetic training batch contains **512 datasets**, arranged as **128 groups of 4 datasets each**. This is a **dataset-generation mechanism**, not part of the model architecture. The batch shares the global constraints coming from the training script and PriorDataset configuration, while the group level is where the code introduces hierarchical parameter sharing rather than sampling everything independently. The repo's hyperparameter sampler is explicitly built around **meta-distributions**—that is, distributions over distribution parameters—so a group does not just share one fixed value, it shares the way values are sampled. For example, one specific group might be governed by a shared recipe like “use an SCM-style prior with 37 features, causal generation enabled, 7 transformation layers, hidden width 24, normal root sampling, target taken as an effect, and non-clique feature/target selection”; the four datasets inside that group would then all be generated under that same broad recipe, so they look related, but they are still four different random datasets because each is a fresh realization. If you also enable subgrouping, you can think of that same 4-dataset group as being split again—for example into **two subgroups of 2 datasets**—where the subgroup shares an even tighter realized generator setup before the final individual datasets are sampled. So the clean mental picture is: **batch = one training-time envelope, group = shared hyperparameter samplers/meta-distributions, subgroup = shared realized generator recipe, dataset = one final sampled synthetic table**

The SCM belongs to the synthetic dataset creation pipeline, not to model training itself. At the conceptual level described in the paper, the prior is a DAG-based SCM: random vectors are sampled at the root nodes, propagated through randomly sampled parent–child transformations, and the final table columns are extracted from randomly assigned nodes in the graph, so the paper-level formulation follows the “sample features and target from within the graph” view rather than a fixed “roots become features and the final node becomes the target” rule. In the repo, this idea is implemented in a simplified layered form through MLPSCM and TreeSCM, so the hierarchy is batch → group → subgroup → dataset, and the SCM is created at the dataset level: `SCMPrior.get_batch()` first builds one parameter dictionary per dataset inside the batch/group/subgroup structure, and `generate_dataset()` then instantiates `MLPSCM(**params)` or `TreeSCM(**params)` and calls it, which means batches, groups, and subgroups share hyperparameters but do not share one common SCM object. ([GitHub](#)) The repo therefore supports both assignment patterns only partially. In MLPSCM, `is_causal=True` makes the generator sample X and y from intermediate hidden states of the transformation chain, with `y_is_effect`, `in_clique`, and `sort_features` controlling whether the target is forced to come from the last layer or can come from earlier states and whether X and y are sampled as a tight block or more independently; by contrast, `is_causal=False` uses the sampled root causes directly as X and the final network output as y . ([GitHub](#)) TreeSCM exposes the same interface in its description, but the current implementation overwrites `is_causal=False` internally, so in practice the tree prior always uses the non-causal pattern in which root causes become the features and the last layer output becomes the target.

In the **repo**, this part is about **synthetic dataset generation**, not about training the final Transformer. Each synthetic dataset is generated by instantiating **one SCM generator**. That generator can be **MLP-based** (`mlp_scm`), **tree-based** (`tree_scm`), or chosen per dataset by `mix_scm`. The important detail is that `mix_scm` does **not** create one graph where some edges use MLPs and others use trees. Instead, the batch generator first chooses a single `prior_type` for each subgroup or dataset-generation recipe, and then `generate_dataset()` instantiates either `MLPSCM` or `TreeSCM` for that dataset. So, in the repo, **one SCM instance uses one generator family only**.

What “**causal**” versus “**direct-prediction**” means is the following. In the MLP generator, the code first samples a set of latent **root causes** using `XSampler`; these are the initial variables before any learned/random transformation is applied. If `is_causal=True`, the MLP is treated like a latent causal chain: the sampled causes are pushed through several hidden layers, and then the final tabular features `X` and target `y` are selected from the **intermediate hidden states** of that chain rather than being tied to only the input or only the output. If `y_is_effect=True`, `y` is taken from the **last hidden outputs**, so the target behaves like a downstream effect; if `y_is_effect=False`, `y` can be taken from earlier hidden states, so it behaves more like an upstream or intermediate variable. If `in_clique=True`, `X` and `y` are sampled from one contiguous block of hidden coordinates, which makes them more tightly related; if `False`, they are sampled more independently from across the hidden representation. By contrast, if `is_causal=False`, the generator stops pretending that features and target are both internal graph variables: it simply uses the sampled root causes directly as `X`, and uses the final network output as `y`. That is why it is called “direct-prediction style”: it behaves like a random function mapping inputs to outputs, rather than like a graph where both inputs and labels are sampled from internal nodes.

The phrase “**how many root causes are used**” refers to the hyperparameter `num_causes`. In causal mode, `num_causes` is the number of latent source variables that `XSampler` generates before the MLP starts transforming them. Those causes are not necessarily the final observed features; they are the hidden starting point from which the observed table is derived. In direct-prediction mode, that distinction disappears, because the code sets `num_causes = num_features`, meaning the sampled root variables themselves become the observed features. In the repo configuration, `num_causes` is not fixed: it is sampled from a log-scaled truncated-normal meta-distribution with lower bound 1, so different synthetic datasets can start from different latent dimensionalities.

For the **MLP-SCM**, the main sampled hyperparameters have very concrete meanings. `num_layers` is the depth of the random MLP; the code requires at least 2 layers, and in causal mode the network is a first linear layer followed by repeated blocks of **activation → linear → Gaussian noise**, while in non-causal mode an extra output block is appended so the last layer

explicitly produces y . `hidden_dim` is the width of the hidden representation, that is, the number of coordinates available at each intermediate layer. In causal mode, the code will automatically enlarge `hidden_dim` if it is too small to provide enough hidden coordinates from which both X and y can be sampled. `mlp_activations` is the activation family used inside those blocks; it is itself sampled from a diverse activation pool via `get_activations(random=True, scale=True, diverse=True)`. `init_std` controls the scale of the Gaussian weight initialization. `block_wise_dropout` chooses between two initialization schemes: either sparse block-wise initialization, where only random blocks of the weight matrix are filled and the rest stay zero, or ordinary dense Gaussian initialization with a Bernoulli mask. `mlp_dropout_prob` is the probability used in that second scheme. `noise_std` controls the Gaussian noise added after each hidden transformation, and `pre_sample_noise_std` decides whether that noise scale is fixed per layer or randomly sampled per output dimension. In the config, these are not drawn once globally; they are produced through the hierarchical sampler, so groups share sampling distributions and subgroups share realized values. `num_layers`, `hidden_dim`, `init_std`, and `noise_std` all come from log-scaled truncated-normal meta-distributions, while `is_causal`, `y_is_effect`, `in_clique`, and `sort_features` are sampled as categorical choices.

For the **tree-SCM**, the basic idea is analogous but the transformation blocks are tree regressors instead of linear layers. A `TreeLayer` takes an input matrix X , creates a random matrix of fake regression targets $y_{\text{fake}} \sim N(0,1)$, fits a tree model to predict y_{fake} from X , and then uses the fitted model's predictions as the transformed representation. So the tree is not trained on a real corrosion label or any real target at all; it is just a way to define a fresh random nonlinear map from one latent node representation to the next. The available tree families are `decision_tree`, `extra_trees`, `random_forest`, and `xgboost`. `max_depth_lambda` and `n_estimators_lambda` are not the depth and ensemble size themselves; they are rate parameters for exponential draws from which the code samples a depth and number of estimators for each layer, then clips them to at most 4 in the current implementation. `hidden_dim` is the output dimension of each intermediate tree layer, just as in the MLP case. `num_layers` is the number of such tree-transformation stages.

There is, however, one very important implementation detail in the current repo: although `TreeSCM` exposes the same interface as `MLPSCM`, inside its constructor it immediately **overrides** several inputs by forcing `is_causal = False`, setting `num_layers = np.random.randint(1, 3)`, and setting `hidden_dim = np.random.randint(3, 10)`. So for the tree generator as currently implemented, the tree prior effectively always runs in **direct-prediction mode**, and the user-specified or hierarchically sampled values of `is_causal`, `num_layers`, and `hidden_dim` are not actually respected in the final instantiated model. That is more precise than saying “the tree prior samples all these parameters normally”: conceptually yes, but in the present code some of them are overridden internally.

The statement about the **initial feature-vector prior** also needs to be phrased carefully. In the repo, the initial latent causes are sampled by `XSampler`, and the allowed modes are

normal, uniform, and mixed. normal means Gaussian causes; if `pre_sample_cause_stats=True`, each cause dimension first gets its own sampled mean and standard deviation and then samples from that personalized Gaussian, otherwise it is standard normal. uniform means each coordinate is sampled uniformly on $[0,1]$. mixed means each cause dimension independently chooses among several marginal families: a Gaussian-like variable, a multinomial/categorical variable, a Zipf-like power-law variable, or a uniform variable. In other words, “mixed” is not one single distribution; it is a heterogeneous column-wise sampler designed to make some latent causes continuous, some heavy-tailed, and some effectively categorical.

The **paper-level TabICLv2 prior** is broader than this repo abstraction. In the paper, the prior is described as a general DAG where root-node random vectors are propagated through node functions, and those node functions can be MLPs, tree ensembles, discretization operators, Gaussian-process functions, linear maps, quadratic maps, EM-like plateau functions, or products of functions. So at the paper level, one graph can contain many different function types across different nodes, whereas the repo abstraction picks one SCM family per dataset. The paper’s “random points” mechanism is also richer than the repo’s XSampler: it can start from standard normal points, points uniform on $[-1,1]^d$, points uniform on the **unit ball**, or normals with random covariance, and only then apply random functions. That is why your earlier “restricted norm distribution” intuition is closer to the **paper** than to the **repo**: the closest paper analogue is “uniform on the unit ball,” which constrains vectors by their norm, while the repo currently exposes only normal, uniform, and mixed at the XSampler interface.

So the clean takeaway is this: in the repo, **each synthetic dataset is generated by one randomly parameterized SCM family**, either MLP-based or tree-based; the hyperparameters control whether the generator behaves like a latent causal graph or like a direct random predictor, how many latent source variables are sampled, how deep and wide the transformations are, how nonlinear and noisy they are, and how the latent root variables themselves are distributed. In the paper, the prior is more general still: it is a modular DAG prior where different node functions and richer root-point samplers can coexist inside one synthetic generative process.

```
pip install -e '[pretrain]'
```

```
Successfully installed annotated-types-0.7.0 charset_normalizer-3.4.7 gitdb-4.0.12  
gitpython-3.1.46 nvidia-nccl-cu12-2.29.7 protobuf-7.34.1 pydantic-2.13.0 pydantic-  
core-2.46.0 regex-2026.4.4 requests-2.33.1 safetensors-0.7.0 sentry-sdk-2.58.0  
smmap-5.0.3 tabicl-2.0.3 tokenizers-0.22.2 transformers-5.5.4 typing-inspection-0.4.2  
urllib3-2.6.3 wandb-0.26.0 xgboost-3.2.0
```

We verified the failure by running `python -m tabicl.train.run --help | sed -n '1,40p'`, which crashed with `ImportError: cannot import name 'TabICL' from 'tabicl'`. Then we inspected the package root with `sed -n '1,40p' src/tabicl/__init__.py` and confirmed that `TabICL` was not exported there, even though `train/run.py` expects from `tabicl import TabICL`.

To fix that local repo mismatch, we added the missing export with `echo 'from .model.tabicl import TabICL' >> src/tabicl/__init__.py`. After that, rerunning `python -m tabicl.train.run --help | sed -n '1,40p'` worked, which confirmed the training CLI is now reachable.

We successfully ran a reduced TabICL smoke test on the BAM GPU system. Using a minimal training configuration, the public training entrypoint generated synthetic prior datasets, trained for 2 steps without errors, and saved checkpoints (`smoke_test_run/step-1.ckpt`, `smoke_test_run/step-2.ckpt`). This confirms that the local environment, patched training entrypoint, GPU execution, and checkpoint writing all work end to end.

```
fcolanto@sv1221:~/projects/tabicl$ python -m tabicl.train.run --wandb_log False --  
wandb_mode disabled --device cuda --prior_device cpu --amp False --max_steps 2 --  
batch_size 4 --micro_batch_size 2 --batch_size_per_gp 1 --min_features 8 --  
max_features 8 --max_classes 3 --min_seq_len 32 --max_seq_len 64 --min_train_size  
0.4 --max_train_size 0.6 --prior_type mix_scm --embed_dim 32 --col_num_blocks 1 --  
row_num_blocks 1 --icl_num_blocks 2 --col_num_inds 16 --checkpoint_dir  
smoke_test_run --save_temp_every 1 --save_perm_every 1000
```

No DDP training

Model has 354699 parameters.

PriorDataset(
 prior_type: mix_scm
 batch_size: 4
 batch_size_per_gp: 1
 features: 8 - 8

```

max_classes: 3

seq_len: 32 - 64

sequence length varies across groups: False

train_size: 0.4 - 0.6

device: cpu

)

No checkpoint found, starting from scratch.

Step: 0%|                                     | 0/2 [00:00<?,
?it/s]/home/fcolanto/projects/tabc1/.venv/lib/python3.12/site-
packages/torch/utils/data/dataloader.py:1118: UserWarning: pin_memory_device is
deprecated, the current accelerator will be used as the device,ignore
pin_memory_device='cuda'.

super().__init__(loader)

Step: 100%|████████████████████████████████████████████████████████████████████████████████| 2/2 [00:04<00:00,
2.03s/it, accuracy=0.552, ce=1.09, prior_time=0, train_time=0.043]

fcolanto@sv1221:~/projects/tabc1$ ls -lah smoke_test_run

find smoke_test_run -maxdepth 2 -type f | sort

total 8.4M

drwxr-xr-x 2 fcolanto domänen-benutzer 56 Apr 14 13:51 .
drwxr-xr-x 12 fcolanto domänen-benutzer 4.0K Apr 14 13:51 ..
-rw-r--r-- 1 fcolanto domänen-benutzer 4.2M Apr 14 13:51 step-1.ckpt
-rw-r--r-- 1 fcolanto domänen-benutzer 4.2M Apr 14 13:51 step-2.ckpt
smoke_test_run/step-1.ckpt
smoke_test_run/step-2.ckpt

```

We then ran a very small GPU smoke test to verify that synthetic prior generation, model training, and checkpoint writing all work on the cluster. The successful command was:

```
python -m tabc1.train.run --wandb_log False --wandb_mode disabled --device cuda --
prior_device cpu --amp False --max_steps 2 --batch_size 4 --micro_batch_size 2 --
batch_size_per_gp 1 --min_features 8 --max_features 8 --max_classes 3 --min_seq_len 32 --
max_seq_len 64 --min_train_size 0.4 --max_train_size 0.6 --prior_type mix_scm --
```

embed_dim 32 --col_num_blocks 1 --row_num_blocks 1 --icl_num_blocks 2 --col_num_inds 16 --checkpoint_dir smoke_test_run --save_temp_every 1 --save_perm_every 1000. The run finished successfully with on-the-fly synthetic prior generation and produced checkpoints smoke_test_run/step-1.ckpt and smoke_test_run/step-2.ckpt. This confirms that the local TabICL setup, patched training entrypoint, GPU execution, and checkpoint saving all function end to end on the BAM infrastructure.

For the real-data evaluation step, the plan is to keep the original corrosion benchmarking workflow (DATACORTECH_model_features...ipynb) unchanged and only replace the model. To make this comparison traceable and reproducible, the TabICL evaluation script was stored inside the local TabICL repository, ideally under `~/projects/tabicl/notebooks/datacortech_tabicl_eval.py`.

To move beyond the smoke-test checkpoint and establish a meaningful local generic-prior baseline, we next prepared a reduced Stage-1-style pretraining run based on the public TabICL training entrypoint. The goal of this run is to generate a locally trained generic-prior model that can later be compared fairly against an informed-prior version under the same downstream Datacortech evaluation protocol. Compared with the full public training recipe, the run was reduced to make it practical for debugging and benchmarking on the BAM GPU system. In addition, the number of features was fixed to 10 in order to match the corrosion-notebook feature subset used in downstream evaluation.

During this process, several code and configuration issues in the current public repository had to be resolved before a stable reduced pretraining run could be launched. First, `python -m tabicl.train.run --help` initially failed because `train/run.py` imports TabICL from the package root, while `src/tabicl/__init__.py` did not export TabICL. This was fixed locally by adding `from .model.tabicl import TabICL` to `src/tabicl/__init__.py`. Second, the reduced training configuration initially failed when `min_seq_len` and `max_seq_len` were both set to 512, because the sequence-length sampler uses `np.random.randint(min_seq_len, max_seq_len)` and therefore requires a strictly larger upper bound. This was resolved by using `min_seq_len=512` and `max_seq_len=513`, which preserves an effectively fixed sequence length of 512 while satisfying the sampler.

A more substantial issue appeared during training itself. Even after fixing the sequence-length setting, the reduced Stage-1-style run failed after several optimization steps with the assertion `d is not supported` when feature grouping is enabled. This revealed a mismatch in the public training path: the training code passes the per-dataset feature-count tensor `d` into the model, while the default column embedder configuration still enables feature grouping, which is incompatible with that input during training. To resolve this, the local copy of

src/tabicl/train/run.py was patched so that the training model is instantiated with `col_feature_group=False`. This is the main deliberate modification made to the original training code. After applying this patch, the reduced generic-prior pretraining job started successfully and progressed through optimization steps without triggering the earlier assertion, indicating that the local training workflow is now functioning beyond the initial smoke-test level.

Train generic:

```
(.venv) fcolanto@sv1221:~/projects/tabicl$ sbatch  
~/projects/tabicl/scripts/train_stage1_mini_generic.sbatch
```

Train informed:

```
(.venv) fcolanto@sv1221:~/projects/tabicl$ sbatch  
~/projects/tabicl/scripts/train_stage1_mini_informed.sbatch
```

Evaluate generic at checkpoint 450:

```
(.venv) fcolanto@sv1221:~/projects/tabicl$ bash  
~/projects/tabicl/scripts/run_datacortech_eval.sh  
~/projects/tabicl/checkpoints/tabicl_s1mini_generic/step-450.ckpt generic_step450  
datacortech_eval_generic_s450
```

Train_stage1_mini_generic.sbatch:

```
#!/usr/bin/env bash

#SBATCH --job-name=tabicl_s1mini_generic
#SBATCH --gpus=1
#SBATCH --mem=32G
#SBATCH --time=12:00:00
#SBATCH --output=/home/%u/tmp/tabicl_s1mini_generic.%j.out
#SBATCH --error=/home/%u/tmp/tabicl_s1mini_generic.%j.err

set -euo pipefail

cd ~/projects/tabicl
source .venv/bin/activate

python -m tabicl.train.run \
    --wandb_log False \
    --wandb_mode disabled \
    --device cuda \
    --dtype float32 \
    --amp False \
    --np_seed 42 \
    --torch_seed 42 \
    --max_steps 100 \
    --batch_size 512 \
    --micro_batch_size 4 \
    --lr 1e-4 \
    --scheduler cosine_warmup \
    --warmup_proportion 0.02 \
```



```
--gradient_clipping 1.0 \  
--prior_type mix_scm \  
--prior_device cpu \  
--batch_size_per_gp 4 \  
--min_features 2 \  
--max_features 100 \  
--max_classes 10 \  
--max_seq_len 1024 \  
--min_train_size 0.1 \  
--max_train_size 0.9 \  
--embed_dim 128 \  
--col_num_blocks 3 \  
--col_nhead 4 \  
--col_num_inds 128 \  
--row_num_blocks 3 \  
--row_nhead 8 \  
--row_num_cls 4 \  
--row_rope_base 100000 \  
--icl_num_blocks 12 \  
--icl_nhead 4 \  
--ff_factor 2 \  
--norm_first True \  
--checkpoint_dir ~/projects/tabicl/checkpoints/tabicl_s1mini_generic \  
--save_temp_every 50 \  
--save_perm_every 250
```

For the generic-prior test, I kept the Stage-1 training configuration aligned with the default settings provided in the public TabICLv2 repository and only reduced the number of training steps to make the run practical for local testing on the BAM GPU cluster. In other words, the generic experiment was intended as a lightweight validation run of the original repository setup rather than a substantially modified training recipe, so the resulting checkpoint can serve as a reference baseline for later comparisons against the informed-prior version.

The first reduced generic-prior training run did not complete successfully. Although training progressed well for most of the run, the cross-entropy loss became undefined at step 61 ($ce=\text{nan}$) and remained unstable in the following steps. The job continued briefly with $ce=\text{nan}$ through step 67, after which the SLURM job was terminated.

To check whether this instability was persistent or batch-specific, I then restarted the generic run from the checkpoint saved at step 50. In this resumed run, training completed the remaining 50 steps without any NaN loss values and finished successfully, ending at 50/50 resumed steps with a final reported cross-entropy of 0.9 and accuracy of 0.633. This suggests that the earlier failure was not necessarily deterministic, since the same training setup could be continued successfully from the intermediate checkpoint.

After the successful restart from step 50, I continued the generic-prior pretraining by repeatedly resubmitting the job from the latest available checkpoint. The subsequent runs loaded checkpoints at steps 100, 150, 200, and 250 and resumed training from those points, which shows that checkpointing continued to work and that the run could be advanced incrementally. However, the loss instability was not fully resolved: in each of these later continuation jobs, the cross-entropy again became undefined at about 61 resumed steps after restart, after which training continued briefly with $ce=\text{nan}$ before the SLURM job was terminated. In practice, this allowed progress up to at least checkpoint step 250, but it also showed that the reduced generic-prior setup remained numerically unstable during longer uninterrupted continuation, rather than failing only in the original first run. Grounded in the uploaded report and SLURM logs: the earlier successful resume from step 50 to step 100 is summarized in the PDF, and the later continuation jobs explicitly loaded checkpoints step-100, step-150, step-200, and step-250, with repeated NaN loss in the corresponding error logs.

Downstream evaluation on the Datacortech benchmark also did not show any meaningful improvement as generic-prior training continued. Across the evaluated checkpoints (steps 50, 100, 150, 200, 250, and 300), the locally trained model produced almost constant low prediction probabilities and, at the default cutoff, classified no positive test samples at all. As a result, all of these checkpoints had test sensitivity 0.0, test specificity 1.0, and test balanced accuracy 0.5, with Cohen's kappa and Matthews correlation both equal to 0. Even when using the cross-validated best cutoff, performance remained essentially flat: CV balanced

accuracy stayed only around 0.51 to 0.514, and test AUROC remained low at roughly 0.52 to 0.54, without any clear upward trend over training. In contrast, the public pretrained TabICLv2 reference model achieved substantially stronger downstream results, with CV balanced accuracy 0.732, test balanced accuracy 0.704, and test AUROC 0.822. Taken together, these evaluations indicate that the reduced locally trained generic-prior checkpoints did not materially improve with further continuation and remained far below the pretrained baseline.

The weak downstream behavior was also confirmed on a separate standard tabular benchmark used as a sanity check. When evaluated on the breast_cancer dataset, the locally trained generic checkpoint at step 300 again performed poorly: cross-validated balanced accuracy remained exactly 0.5, holdout balanced accuracy was also 0.5, and holdout AUROC reached only 0.529. By contrast, the public pretrained TabICLv2 reference model performed strongly on the same task, with cross-validated balanced accuracy 0.977, holdout balanced accuracy 0.974, and holdout AUROC 0.996. This shows that the problem was not limited to the Datacortech corrosion benchmark; even on a standard off-the-shelf dataset used only for checking, the reduced local generic-prior model still failed to reach competitive classification performance.

To see whether the later curriculum stage would stabilize or improve the locally trained model, I then took the last available reduced Stage-1 generic checkpoint (step 300) and used it to initialize a reduced Stage-2 run, following the intended sequential Stage-1 → Stage-2 training setup of TabICLv2. In this run, the model was started in Stage-2 mode with only the model weights loaded from the Stage-1 checkpoint, distributed training over two GPU processes, per-GPU batch size 256, batch_size_per_gp=2, and a much broader synthetic sequence-length range of 1,000 to 40,000. However, the run immediately showed severe memory pressure: many individual micro-batches triggered OOM warnings and were skipped throughout training. The job progressed only to step 39/300 before one worker process was killed and the SLURM step reported OOM-kill events, after which the Stage-2 run terminated. In other words, promoting the last reduced Stage-1 checkpoint into Stage 2 did not resolve the instability and instead exposed an additional scaling problem, namely that the Stage-2 long-context setup was not yet stable under the current reduced local training configuration. The fact that Stage 2 is meant to continue from Stage 1 is also consistent with the earlier project notes describing the TabICLv2 curriculum as sequential.

I also evaluated the short Stage-2 continuation checkpoint (step 30) to see whether moving into the next curriculum stage would improve downstream behavior before the run failed. It did not. On the Datacortech benchmark, the Stage-2 generic checkpoint again produced an almost constant score range (test probabilities about 0.361 to 0.376), predicted no positive

test samples at the default cutoff, and therefore remained effectively at chance in balanced terms, with test positive rate 0.0 and cross-validated balanced accuracy only 0.511 at the best cutoff. This was still far below the public pretrained TabICLv2 reference, which on the same evaluation reached CV balanced accuracy 0.732 and test AUROC 0.822. The same pattern appeared on the additional breast_cancer sanity-check dataset: the Stage-2 generic checkpoint achieved CV balanced accuracy 0.5, holdout balanced accuracy 0.5, and holdout AUROC only 0.586, whereas the pretrained reference again performed strongly, with CV balanced accuracy 0.977 and holdout AUROC 0.996. Thus, even after briefly continuing the last local Stage-1 checkpoint into Stage 2, the resulting model still did not show meaningful downstream improvement, either on the corrosion benchmark or on the standard tabular check dataset. The Stage-1 → Stage-2 progression itself is consistent with the sequential TabICLv2 curriculum described in the project notes

To assess whether the earlier instability was mainly caused by an overly aggressive optimizer setting, I repeated the reduced generic-prior training run with a lower learning rate, reducing lr from $1e-4$ to $5e-5$ while otherwise keeping the setup unchanged. The run still failed, indicating that the instability is unlikely to be explained by learning rate alone. Instead, the result suggests that other aspects of the reduced Stage-1-style configuration, such as the breadth of the synthetic prior or the overall training setup, are more likely contributing to the observed failures. The original reduced generic run had used $lr=1e-4$ and had previously become unstable when the cross-entropy loss turned into NaN after progressing normally through the early steps.

For the next generic-prior experiment, I switched to a more task-matched configuration. Compared with the earlier reduced Stage-1-style run, the total batch size was reduced from 512 to 64, the training budget was increased from 100 to 1000 steps, the prior was changed from mix_scm to mlp_scm, the feature count was fixed at 10 to match the Datacortech subset, the train-size range was narrowed from 0.1–0.9 to 0.6–0.8, and the warmup proportion was increased from 0.02 to 0.05. Within this revised setup, I first trained with $lr=5e-5$ and then repeated the experiment with a lower learning rate of $5e-6$ to test whether further reducing the optimizer step size would improve behaviour. The lower-learning-rate run remained stable and did not show the earlier NaN-loss failure, but it also did not produce a clear qualitative improvement over the higher-learning-rate version. Downstream evaluation of the checkpoint at step 350 also remained weak: the locally trained generic model achieved a CV balanced accuracy of about 0.58 and a test balanced accuracy of about 0.58 (AUROC about 0.52), which was substantially below the public pretrained TabICLv2 checkpoint on the same evaluation protocol. Overall, these results suggest that lowering the learning rate improved stability, but did not by itself yield a better downstream generic baseline.